本章路線

這章要回答的核心問題是：系統不確定資源何時釋放時，什麼條件足以造成永久等待，如何用可負擔的策略處理？

前置知識。 process state、mutex 與 semaphore、向量的逐分量比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

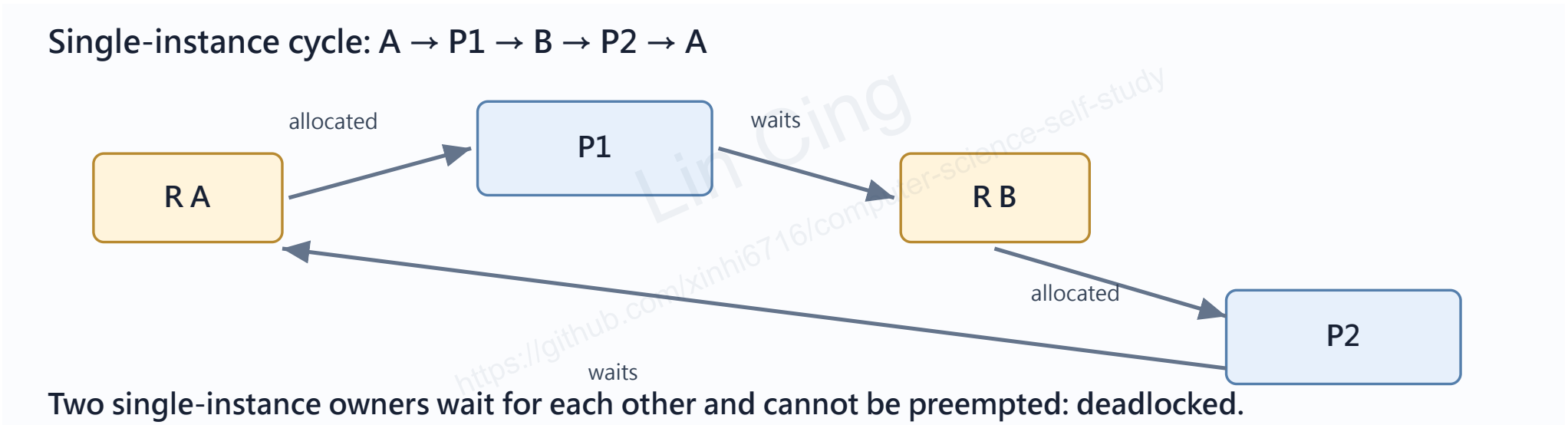
學習目標

- 證明必要條件不等於充分條件
- 設計破壞 circular wait 的 lock ordering
- 以 Banker's algorithm 檢查 safe state

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 系統模型

- 資源型別有固定 instance 數；process 對資源執行 request、use、release。request 可能立即成功，也可能在資源不足時阻塞。
- 資源配置圖中 process node 指向 resource 表示等待，resource instance 指向 process 表示持有。圖模型只描述等待與持有關係。



7.1　模型、時序與反例：系統模型

模型假設。 資源 instance 數固定，持有與等待關係可原子記錄；process 不會暗中釋放 resource。

不變量與時序。 每個 resource instance 最多被一個 process 持有；wait edge 與 allocation edge 必須對應真實 request/hold。

反例或邊界。 圖上看似兩條 waiting edges，但其中一個 resource 有額外未被追蹤的 instance，仍可能前進。

完整算例。 資源 R1 有 1 instance、被 P1 持有；R2 有 1 instance、被 P2 持有。P1 等 R2，P2 等 R1，wait-for graph 有 P1→P2→P1 循環且沒有非 deadlocked holder 可釋放，因此兩者 deadlocked。若 R2 另有 instance 被 P3 持有且 P3 可執行，圖仍可有 cycle，但系統不一定 deadlocked。

思考題。 多 instance 圖中，哪項額外事實能打破看似循環的等待？

7.1　資源、狀態與 deadlocked set

resource type 是一組功能等價的 instance。若兩個 printer 可互換，它們屬於同一 type；若位置或功能不可互換，就必須分成不同 type。process 對每個 resource 依序執行 request、use、release。

request 可以立即成功，也可以在 instance 不足時等待。system table 至少保存：

- 每個 type 的 instance 總數。
- 每個 instance 的 free 或 allocated 狀態。
- allocated instance 的 owner。
- 每個 waiting request 的 requester 與 resource type。

一個 process set 處於 deadlocked state，表示集合內每個 process 都在等待一個事件，而該事件只能由集合內另一個 waiting process 產生。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1　兩個已死結的具體狀態

同一 resource type。 三個 CD-RW 共 3 個 instance，P1、P2、P3 各持有 1 個，接著各再請求 1 個。Available=0，且沒有任何 owner 能完成並 release，三者形成 deadlock。

不同 resource types。 系統有 1 個 printer 與 1 個 DVD。P1 持有 DVD 並等 printer，P2 持有 printer 並等 DVD。此狀態滿足：

Process	Allocation	Request
P1	DVD	printer
P2	printer	DVD

資源種類不同不影響 deadlock；判斷依據是 request/hold 關係與是否仍有可完成的 transition。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.2 死結特徵

- deadlock 需要同時成立的四個必要條件：mutual exclusion、hold and wait、no preemption、circular wait。必要條件是判定線索，不自動證明死結。
- cyclic wait 可能只是暫時且可被其他 process 釋放資源打破；多 instance 圖還需確認是否能由其他 instance 前進。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.2　模型、時序與反例：死結特徵

模型假設。 四個條件針對不可共享、可持有等待、不可搶占與循環等待；未來資源釋放假設尚未發生。

不變量與時序。 真正 deadlock 需所有參與者都無法在有限步內前進，不能只用 cycle 存在代替。

反例或邊界。 多 instance 情形：R1 有 1 份由 P1 持有，R2 有 2 份分給 P2/P3；P1 等一份 R2、P2 等 R1，但 P3 可完成並釋放 R2，因此圖中 P1/P2 的環不代表已死結。若 A/B 都是單一 mutex，不能用額外持有者的說法解開同一環。

完整算例。 固定 lock order F(A)=1、F(B)=2。P1 取得 A 後請求 B 合法；P2 若先取得 B 再請求 A，runtime order checker 在第二次 acquire 前拒絕並回報 expected A<B、actual B→A，P2 不會進入等待 A 的佇列，但拒絕本身不會釋放已持有的 B；呼叫端必須明確釋放 B、恢復一致狀態，再依 A→B 重試。若只忽略錯誤並永遠保留 B，P1 仍可能永久等待。

思考題。 必要條件和檢測快照各缺少哪一種資訊？

7.2　四個必要條件

條件	在此題中的含義	若持續成立
Mutual exclusion	至少一個 resource 不可同時共享	其他 requester 必須等
Hold and wait	process 已持有至少一個 resource，又等待另一個	等待可能連結兩份 ownership
No preemption	resource 不能被系統強制收回，只能持有者自願 release	外部不能直接拆環
Circular wait	`P0→P1→...→Pn→P0` 的等待鏈	每個參與者的下一步依賴下一人

circular wait 已蘊含 hold and wait，因此四個條件不是彼此邏輯獨立。四者同時成立仍只是 deadlock 的必要條件，不能用來斷言目前已經無限期卡住。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.2 resource-allocation graph 的判準

process node 為圓形，resource type 為矩形，instance 為矩形內圓點。

- request edge `Pi→Rj` 表示 Pi 正在等待 Rj 的 instance。
- assignment edge `Rj→Pi` 表示 Rj 的某個 instance 已分配給 Pi。
- 若 request 立即成功，request edge 立即轉成 assignment edge。
- release 時刪除對應 assignment edge。

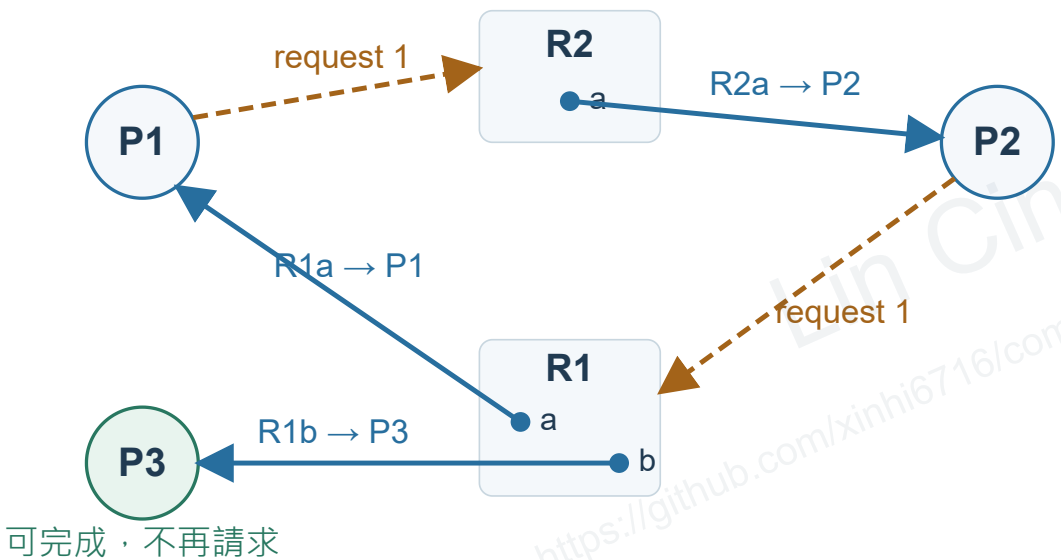
定理。 若 graph 無 cycle，系統必無 deadlock。若 cycle 的每個 resource type 都只有一個 instance，cycle 是 deadlock 的充分且必要條件。若 cycle 涉及多 instance type，cycle 只保證可能存在 deadlock，仍需檢查其他 instance 是否可令某 process 完成。

https://github.com/xinhi6716/computer-science-self-study

7.2　有環，為什麼仍能完成？

R1 有兩個可互換的實例 R1a、R1b，R2 只有 R2a。三個 process 都遵守「取得目前請求後，即可完成並歸還全部持有資源」；P3 不再請求資源，可直接完成。初始沒有空閒實例。

初始狀態：Total=(2,1)，Available=(0,0)



箭頭從誰指向誰？

實線：實例 → 持有者

虛線：等待者 → 資源型別

R1 的兩個點是兩份獨立實例。

環：P1 → R2 → P2 → R1 → P1

P3 不在環上，卻持有另一份 R1。

先讓 P3 完成，就能解開等待。

環上的 P1、P2 目前都在等待，但 R1b 的持有者 P3 並未等待。**有環不等於所有可解環的釋放途徑都被堵住。**

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.2　先完成 P3，再依序解除等待

同一個完成順序 P3 → P2 → P1；向量順序為 (R1,R2)

事件	R1a owner	R1b owner	R2a owner	Available
初始：P1、P2 等待	P1	P3	P2	(0, 0)
P3 完成，歸還 R1b	P1	空閒	P2	(1, 0)
授予 P2：R1b	P1	P2	P2	(0, 0)
P2 完成，歸還 R1b、R2a	P1	空閒	空閒	(1, 1)
授予 P1：R2a	P1	空閒	P1	(1, 0)
P1 完成，歸還 R1a、R2a	空閒	空閒	空閒	(2, 1)

授予資源會減少 Available，真正完成並歸還才會增加。只寫完成順序 P3→P2→P1，會省略 P2、P1 取得請求時的兩次扣除；上圖把這些中間狀態一併列出。

7.2　Detection 的 Work 是模擬帳，不是即時庫存

向量順序固定為 (R1,R2)。目前尚未滿足的 Request 與既有 Allocation 為：

Process	Allocation	Request	檢查前 Work	模擬完成後 Work
P3	(1,0)	(0,0)	(0,0)	(1,0)
P2	(0,1)	(1,0)	(1,0)	(1,1)
P1	(1,0)	(0,1)	(1,1)	(2,1)

每列先檢查 Request≤Work；若可滿足，再令 Work←Work+Allocation。它把「先授予 Request，之後歸還 Allocation+Request」合成一步，所以淨增加只有 Allocation。Work 中的實例可由已模擬完成者歸還，**不代表那些 process 已真的執行；初始 Available 仍是 (0,0)**。

此處比較的是**目前未滿足的 Request**，不是 Banker's algorithm 的最大剩餘需求 Need。完整 detection 程序見 7.6。

7.2　只改 P3 的下一步，就會真的死結

保留圖中的全部持有關係與 P1、P2 的請求，只把 P3 改成「也必須再取得一份 R2 才能完成」，即新增 P3→R2。

初始狀態	P3 的 Request	Work=(0,0) 時可先完成者	結果
原例	(0,0)	P3	完成順序 P3→P2→P1
改例	(0,1)	無	P1、P2、P3 皆死結

改例的 P1 與 P3 都等待 P2 持有的 R2a；P2 等待的 R1，兩份都在這兩個等待者手上。所有實例都困在等待集合內，沒有能先歸還的持有者。此結論假設資源不可搶占、沒有取消或額外實例，也沒有完成前的提前釋放。單純把 P3 暫時移出 CPU 不會改變原例的資源可行性；排程飢餓與資源死結須分開判讀。

7.2　lock deadlock 的可重現交錯

thread A 依序取得 `first`、`second`；thread B 依序取得 `second`、`first`。

時刻	動作	持有
t0	A 取得 first	A:first
t1	B 取得 second	A:first; B:second
t2	A 等 second	A 持 first 等 B
t3	B 等 first	B 持 second 等 A

四條件皆成立且沒有非 waiting holder，兩 thread 永久等待。若 A 在 t1 前完成全部工作並 release，這個特定交錯不會發生。deadlock 需要特定相對時序，單次壓力測試不能證明不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.3 處理方法

- prevention 靜態破壞一個必要條件；avoidance 在每次配置前用未來宣告資訊檢查安全；detection/recovery 則允許形成後再處理。
- 完全忽略 deadlock 是某些一般用途系統的成本選擇，不代表應用程式沒有鎖順序與 timeout 設計問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.3　模型、時序與反例：處理方法

模型假設。 比較策略需知道靜態需求、runtime overhead、恢復資料成本與是否拒絕請求。

不變量與時序。 任何允許的配置仍符合所選策略的 deadlock 保證；避免策略不能暗中依賴未宣告需求。

反例或邊界。 Avoidance 使用錯誤的最大需求，演算法仍可能判 safe，但 runtime 請求會超出模型。

完整算例。 固定 R1、R2 各一個 instance。prevention 若用 F(R1)=1<F(R2)=2，P1 持 R1 時只能再請求 R2；沒有反向 acquire 路徑。avoidance 先記錄 P1、P2 的 claim edges，P2 若提出會形成 cycle 的 request，就在 grant 前等待。detection 允許配置先發生，週期掃描看到 wait-for graph cycle 後才選 victim；三者對同一 request 的時點與資訊需求不同。

思考題。 策略的資訊需求與失敗代價是哪一個更早決定設計？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.3　四種處理策略的資訊需求

策略	採用的資訊	保證	主要成本
Prevention	只靠限制 request 的協議	至少破壞一個必要條件，理論上不進入 deadlock	低利用率、拒絕 request、rollback
Avoidance	process 事先宣告最大需求	每次配置後仍存在 safe sequence	需要可信 claim，runtime safety test
Detection/recovery	runtime allocation、request 與 snapshot	允許形成 deadlock，之後找出並恢復	資訊維護、偵測延遲、victim 損失
Ignore	不維持 deadlock protocol	不提供一般 deadlock 保證	卡住時由應用、timeout 或重啟處理

第一與第二種都避免「系統進入無法完成全體工作的狀態」，但 prevention 限制 request 形狀，avoidance 則根據未來需求判斷一個目前有空閒的 request 是否仍該等待。

7.4 Deadlock prevention

- mutual exclusion 對不可共享資源難移除；hold-and-wait 可要求一次取得全部或先釋放持有；no preemption 可強制搶回；circular wait 可建立全域資源順序。
- 一次取得全部可能降低利用率；強制搶回需要保存與恢復 process 狀態；lock ordering 最容易在軟體工程中驗證，但包含動態資源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.4.1 Lock ordering 不變量

- 若每條執行路徑都只依遞增全域順序取得鎖，等待邊永遠由低階指向高階，無法形成嚴格循環。
- 例外路徑、callback、condition wait 與升級鎖也必須遵守順序；只在主路徑排序不足。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.4　模型、時序與反例：Deadlock prevention

模型假設。 資源與 lock order 已知，所有取得路徑包含 callback、升級與例外；no preemption 的狀態可恢復。

不變量與時序。 lock order 若嚴格遞增，時序中的等待邊無法形成有向環；資源必須在釋放前恢復一致。

反例或邊界。 主路徑 A→B、補救路徑 B→A，仍能形成 deadlock，雖然大多數正常路徑有排序。

完整算例。 A/B 兩 locks 的固定順序 A→B。Thread1 acquire A 後 acquire B；若 Thread2 已持 B，Thread1 等待 B。Thread2 再請求 A 時，order checker 在阻塞前拒絕。Thread2 應恢復一致狀態、釋放 B，再依 A→B 重試；如此不會建立反向等待邊，Thread1 也能繼續。拒絕請求本身不會自動釋放 B。

思考題。 如何把「全域順序」擴展到動態名稱資源？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.4　逐一破壞必要條件

Mutual exclusion。 將 resource 改為可共享，例如多人同時讀取 read-only file。mutex 對受保護的可寫資料本質上不可共享，所以這條通常只能局部使用。

Hold and wait。 要求啟動前一次取得全部 resource，或每次 request 前先 release 所有 resource。前者可能讓印表機閒置到程式最後；後者可能破壞尚未持久化的中間資料。

No preemption。 新 request 無法滿足時，收回該 process 現有 resource，待舊與新資源都可重新配置後 restart；或只搶收回應中 waiting process 的資源。只有 state 可保存與重建的資源適合此做法。

Circular wait。 為所有 resource type 指定嚴格順序，只能向較高順序 request。這是實作上最常見的破壞方式，代價是違反順序的 request 必須改成等待或重新設計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.4 resource ordering 的反證法

令 $F: R \to N$ 為一對一順序。協議要求 process 持有 R_i 時，只能再取得 R_j，且 $F(R_i) < F(R_j)$。

假設仍有 circular wait，沿等待邊列 resource 會得到

$$F(R_0) < F(R_1) < \cdots < F(R_n) < F(R_0)$$

由遞移性得到 $F(R_0) < F(R_0)$，矛盾。因此不可能存在嚴格順序下的 cycle。

動態鎖的實際版本。 兩帳戶 from/to 的管理鎖不能按參數順序取得。先對帳戶 id 排序，再依小 id 到大 id 取得兩鎖；release 可反向或按 API 規則，但新的 acquire 必須維持順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.5 Deadlock avoidance

- Banker's algorithm 假設 process 事先宣告每類資源最大需求、持有與仍可請求量已知，且 process 不超過宣告上限。resource allocation graph 的 request/claim edge 用來判斷 future safe。
- safe state 存在至少一個完成順序，使每步需求都能由目前 available 資源滿足。safe 不代表現在每位 process 都不用等待；unsafe 也不代表 deadlock 已發生。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.5　模型、時序與反例：Deadlock avoidance

模型假設。 每個 process 的 maximum claim 是上界、最大需求可達、資源數量固定，且 snapshot 在安全點取得。

不變量與時序。 安全狀態存在完成序列，使每次 Need_i <= Work；每完成一個 process，Work 增加其 Allocation。

反例或邊界。 已無 process 可開始的 unsafe state 仍可能未死結，若某 process 自願釋放全部；avoidance 只給保守保證。

完整算例。 課本 T0 的 Available=(3,3,2)，依 Need<=Work 選 P1，Work 成 (5,3,2)；再選 P3、P4、P0、P2，Work 依序成 (7,4,3)、(7,4,5)、(7,5,5)、(10,5,7)，證明 <P1,P3,P4,P0,P2> 是 safe sequence。

思考題。 unsafe 與 deadlocked 的邏輯關係是什麼？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.5 單 instance avoidance 的 claim edge

單 instance graph 額外畫 claim edge `Pi→Rj`，表示 Pi 未來可能 request Rj。request 到來時，claim edge 先轉 request edge；若 grant，再轉 assignment edge。

grant 前要檢查：把提出的 request edge 轉為 assignment edge 後，graph 是否新增 cycle。若無 cycle，保留分配；若有 cycle，Pi 等待，狀態不變。

claim edge 必須在 process 開始前已知，或只能在該 process 尚無 assignment edge 時加入。若未來需求宣告不完整，cycle 檢查無法覆蓋真實 request。cycle detection 可用 adjacency matrix 的 `O(n^2)` 做法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.5 Banker 的資料與 safety 規則

令 n 為 process 數，m 為 resource type 數，`Allocation_i` 是 Pi 現有資源，`Need_i` 是完成前最多還會請求的差額。

```
Need = Max - Allocation
```

safe sequence 的每一步都要求 `Need_i <= Work`。完成 Pi 後：

```
Work = Work + Allocation_i
Finish[i] = true
```

此步驟表示 Pi 能取得它宣告的剩餘需求，完成任務後把全部已配置與新取得資源返回。若所有 Finish 都能變成 true，當下 state 是 safe；否則 unsafe。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.5 Safety algorithm的一般正確性

假設每個process的Max宣告合法、所有resource分量非負、process取得Need後能在有限時間完成並歸還全部資源。Work起始Available；若某Pi的Need_i<=Work，先給它Need，再收回Allocation_i+Need_i，淨變化是Work−Need_i+(Allocation_i+Need_i)=Work+Allocation_i。故完成一步不會減少其他process未來可用的資源，也不把Need平白加進總量。

若存在某安全完成序列，而演算法先挑了序列中不是第一個的可完成Pi，可以先完成Pi再維持其餘原順序、略過原來的Pi位置。其餘每一步面對的可用資源只會更大，不會較小；因此任選一個Need<=Work不會破壞仍存在的安全序列。這說明greedy選擇不需對每個候選分支回溯。

若演算法停止但仍有unfinished，反設原state有安全序列，取該序列中第一個尚未被演算法完成的Pj。它在該序列之前的process都已在演算法的完成集合內；當前Work包含這些Allocation，且可能更多，所以Need_j必<=當前Work，與停止時沒有候選矛盾。故找不到候選而仍有unfinished便unsafe；若全部完成，記錄的順序本身就是安全證書。

7.5 resource-request 演算法

Pi提出m維Request_i時，先要求每個分量都是非負整數且維度正確；本節x<=y一律表示每個j都有x[j]<=y[j]。其否定是「存在至少一個分量較大」，不是「每個分量都較大」。

1. 若存在j使Request_i[j]>Need_i[j]，拒絕並視為超過最大宣告。
2. 否則若存在j使Request_i[j]>Available[j]，Pi等待現有資源。
3. 暫時令 `Available-=Request`、`Allocation_i+=Request`、`Need_i-=Request`。
4. 對 trial state 執行 safety algorithm。
5. safe 才真正授予；unsafe 要撤銷 trial 修改，讓 Pi 等待。

反例：Total=(2,1)、唯一Pi的Allocation=(1,0)、Need=(1,1)、Available=(1,1)是合法state。Request=(2,0)在第一維同時超過Need與Available，即應在第1步error；若把向量>誤解為兩維都大，會錯放而得到Available=(−1,1)，已違反非負資源。

完整流程的必要性是：第 3 步是 transaction，不能在 unsafe 後留下部分扣款或部分 allocation。每輪 safety 至多檢查 n 個 process，可做 n 次，因此典型複雜度為 `O(m n^2)`。

7.5 Banker T0 完整矩陣

Total=`(10,5,7)`，Available=`(3,3,2)`。

Process	Allocation	Max	Need=Max-A
P0	(0,1,0)	(7,5,3)	(7,4,3)
P1	(2,0,0)	(3,2,2)	(1,2,2)
P2	(3,0,2)	(9,0,2)	(6,0,0)
P3	(2,1,1)	(2,2,2)	(0,1,1)
P4	(0,0,2)	(4,3,3)	(4,3,1)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.5　Banker T0 完整矩陣 （續）

Safety trace：

完成順序	Work 轉移
初始	(3,3,2)
P1	(5,3,2)
P3	(7,4,3)
P4	(7,4,5)
P0	(7,5,5)
P2	(10,5,7)

所以 `<P1,P3,P4,P0,P2>` 是 safe sequence。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.5 P1 request 與 unsafe 拒絕

P1 提出 `Request1=(1,0,2)`：

1. `(1,0,2)<=Need1=(1,2,2)`，未超過 claim。
2. `(1,0,2)<=Available=(3,3,2)`，可做 trial。
3. trial 後 Available=`(2,3,0)`、Allocation1=`(3,0,2)`、Need1=`(0,2,0)`。

Trial safety 可依 `P1→P3→P4→P0→P2` 完成，因此授予此 request。

反例：在上述 trial 後，P0 請求 `(0,2,0)`。資源量足夠，但 trial 後 Work=`(2,1,0)`，沒有任何 P0 到 P4 的 Need 小於等於 Work，safety 失敗，因此 P0 必須等待。

unsafe 不代表此刻已 deadlock。若 process 自願提前釋放資源，仍可能完成；avoidance 的保守保證是「不能保證存在完成序列」，不是「必然永久卡住」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6 Deadlock detection

- single instance 可用 wait-for graph 的 cycle 判定；多 instance 要用可用量逐步縮減需求並檢查是否能完成所有 process。
- detection 的頻率是 overhead 與偵測延遲的取捨。演算法判定目前 snapshot，不代表未來永不 deadlock。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6　模型、時序與反例：Deadlock detection

模型假設。 wait-for graph 或 available/request matrices 的 snapshot 一致，資源所有權與 request 可被原子觀察。

不變量與時序。 single-instance cycle 表示 deadlock；多 instance 必須反覆找 Request_i <= Work 的 process 並將資源加回。

反例或邊界。 若快照跨越兩次資源釋放，可能把已完成的 work 誤認成未可用，或遺漏等待。

完整算例。 Work=(0,0,0)，P0 Request=(0,0,0) 先完成，加回 Allocation0=(0,1,0) 使 Work=(0,1,0)；之後若 Request P3=(1,0,0) 仍不可完成，就保留 P3 的 Finish=false。只有當另一 process 的 Request<=Work 並加回 Allocation 後，後續等待才可能被解除。

思考題。 何種 snapshot consistency 假設能讓 reduction 結果可信？

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.6　單 instance：wait-for graph

移除資源節點，保留 process 節點。若存在 request edge `Pi→Rq` 與 assignment edge `Rq→Pj`，合併成 `Pi→Pj`。此邊表示 Pi 等 Pj release Rq。

若每個 resource type 都只有一個 instance，deadlock 存在若且唯若 wait-for graph 有 cycle。cycle 上的每個 process 都不能靠 cycle 外的 instance 自行解除等待。若任何 cycle 經過一個多 instance resource，就必須回到 allocation/request 狀態做可完成性分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6　多 instance detection 演算法

輸入 Available、Allocation、Request。

```
Work = Available
Finish[i] = (Allocation_i == 0)

while exists i:
    Finish[i] == false and Request_i <= Work:
        Work = Work + Allocation_i
        Finish[i] = true

if any Finish[i] == false:
    Pi is not shown able to complete in this snapshot
```

若某 process 的目前 request 能由 Work 滿足，演算法就樂觀假設它不再提出更多 request，完成後歸還全部 Allocation。若它在下次偵測前又提出更多 request，新的 snapshot 會再判斷。複雜度同為 O(m n^2)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.6　完整 reduction 例

T0 的 Available=`(0,0,0)`。

Process	Allocation	Request
P0	(0,1,0)	(0,0,0)
P1	(2,0,0)	(2,0,2)
P2	(3,0,3)	(0,0,0)
P3	(2,1,1)	(1,0,0)
P4	(0,0,2)	(0,0,2)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6　完整 reduction 例 （續）

完成	Work 加回 Allocation 後
P0	(0,1,0)
P2	(3,1,3)
P3	(5,2,4)
P1	(7,2,4)
P4	(7,2,6)

所有 Finish 為 true，因此 snapshot 沒有 deadlock。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6　P2 多請求後的 deadlocked set

修改 P2 Request=`(0,0,1)`。Work=`(0,0,0)`：

1. P0 Request=`(0,0,0)` 可完成，Work 變成 `(0,1,0)`。
2. 其他 Request 都不能由 Work 滿足。
3. Finish P1、P2、P3、P4 保持 false。

因此此snapshot的不可完成集合為 `{P1,P2,P3,P4}`。前面的P0完成只是偵測演算法的**模擬**，不會真的把資源放回Available。若P0尚未實際完成，終止P1並安全回收其Allocation=(2,0,0)後，真實Available由(0,0,0)變成(2,0,0)，不是(2,1,0)。重新以Work=Available開始，對存活者做reduction：

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6　P2 多請求後的 deadlocked set （續）

模擬完成者	當前Request	加回Allocation後的Work
初始，P1已終止	—	(2,0,0)
P0	(0,0,0)	(2,1,0)
P3	(1,0,0)	(4,2,1)
P2	(0,0,1)	(7,2,4)
P4	(0,0,2)	(7,2,6)

每列先驗Request≤Work，完成後只加回原Allocation。這個序列證明存活者可完成；只有它們真的執行並歸還資源，真實Available才依序增加。若P0早已實際完成，終止P1後才可直接由Available=(2,1,0)開始。

偵測頻率越高，越早找出最接近最後等待的 requester，但執行成本也越高；固定 interval 可能一次看到多個 cycle。

7.7 Recovery

- recovery 可終止全部或部分 processes，或 preempt 資源。victim 選擇需考慮工作量、資源、優先權與重複受害。
- 終止後必須恢復被暫停 process 的一致狀態。若不記錄 rollback point，只奪回 lock 可能留下半完成資料結構。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.7　模型、時序與反例：Recovery

模型假設。 選定 victim 後可 rollback 到交易或 checkpoint，所有 lock/resource 的 ownership 可重建。

不變量與時序。 恢復後共享資料回到一致版本；同一 process 不得被無限次選為犧牲者。

反例或邊界。 只釋放鎖而不 rollback 已部分寫入的表格，下一個持有者會看到不一致的資料。

完整算例。 兩個 deadlocked transactions 中，T2 的 checkpoint cost=2、rollback count=0，T4 的 cost=3、rollback count=0。先選 T2，rollback 到 checkpoint 並釋放2 resources；detection 重跑後若 T4 可完成就停止犧牲。T2 的 rollback_count 增為1，使其下一次 victim cost 提高。

思考題。 如何證明 recovery 沒有讓使用者看到虛假的成功？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.7　終止 process 的恢復

全部終止。 一次 abort 所有 deadlocked processes，保證拆環，但損失全部未完成工作。

逐一終止。 每次 abort 一位 victim，回收其全部 resource，再重跑 detection。未終止者若因資源返回而可進展，就停止犧牲。

victim 的選擇是 policy：

- priority 與互動程度。
- 已執行時間、剩餘估計時間。
- 持有與需要的 resource 數量。
- 資源是否容易 reclaim。
- 需要終止多少個 process 才能拆環。

若 process 正在修改 file 或驅動裝置，直接 abort 可能留下外部半完成狀態。恢復設計要把 rollback、journal 或 device reset 納入成本，不能只把 lock 視為可回收物件。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.7 resource preemption 與 starvation

resource preemption 要回答三個獨立問題：

1. **Victim**。 要收回哪位 process 的哪些 resource，成本如何計算。
2. **Rollback**。 被收回者要回到哪個一致狀態。最容易是全部 abort 後 restart；較省成本的是回到 checkpoint，但需要保存足夠 undo state。
3. **Starvation**。 同一 process 不能永遠因 cost 最低而被選為 victim。

`cost_i = execution_i + rollback_count_i × k`（k>0）會提高反覆回滾者的相對成本，是減少重複選取的啟發式。只有在競爭集合與其他候選成本受適當限制時，才可推導它何時不再最低；任意新到達或成本變動下，不能據此保證公平。明定每個 process 最多連續被選 r 次（r 有限）可限制連續受害次數，但仍須交代沒有其他可選 victim 時的恢復策略；此局部限制也不等於已證明全體最終完成。有限等待或完成保證須由完整 workload 與選取規則推導。

7.8 總結

- 四必要條件提供 prevention checklist；資源配置圖與 safety algorithm 提供不同資訊假設下的分析。
- deadlock 與 starvation 不同。deadlock 是等待環中沒有參與者可前進；starvation 是系統可前進但某個體無限期被跳過。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.8　模型、時序與反例：總結

模型假設。 deadlock、unsafe、cyclic wait 與 starvation 是不同狀態，檢測策略必須說明資源 instance 與時序。

不變量與時序。 最終策略要在每條配置與恢復路徑上維持資源守恆，並有明確 progress 假設。

反例或邊界。 lock ordering 可避免某些循環，但若持鎖 thread 無限期餓死，process 仍可能無法完成。

完整算例。 單 instance 的 P1/P2 互等形成 wait-for cycle，且兩人都持有對方所需資源，因此 deadlocked。另一個獨立 trace 中，P3 未被選而 P4 已執行 5 次，只能顯示 starvation 風險，不能由有限觀察證明沒有等待上界。明確反例採單 CPU、零切換成本、固定優先權且不 aging：P3 在 t=0 到達，priority=1、burst=1；對每個整數 $k \geq 0$，P4_k 在 t=k 到達，priority=2、burst=1。每個整數時刻先處理完成及到達，再選最高 priority。t=0 選 P4_0；若 t=k 選 P4_k 跑到 k+1，下一個 P4_(k+1) 正好到達且又優先於 P3。歸納可知每個 [k,k+1) 都執行 P4_k，P3 對任意 k 都仍 ready 而未取得 CPU；系統持續完成工作但 P3 無限等待，這才建立 starvation。

思考題。 哪個案例可以同時沒有 deadlock 但仍有 starvation？

7.8　deadlock 判斷的完整決策表

你觀察到什麼	可以先得到什麼	還需要什麼
四必要條件成立	deadlock 可能	是否存在無可進展 transition
single-instance wait-for graph 無 cycle	未 deadlocked	不需再做 cycle 充分性判斷
single-instance graph 有 cycle	deadlocked	確認所有 cycle resource 都是單 instance
multi-instance graph 有 cycle	deadlock 可能	availability/reduction 或 safety 分析
unsafe state	avoidance 不能提供完成保證	實際 request 可能尚未造成 deadlock
某 process 長期未執行但系統持續前進	starvation 可能	公平 policy 與等待上界

deadlock 是等待集合沒有可完成的內部 transition。starvation 是系統仍有 transition 與進展，但某個體沒有 finite wait bound。兩者可以同時存在，也可以各自獨立發生。

本章收束

- 必要條件不能單獨證明充分
- avoidance 依賴最大需求宣告
- 檢測、恢復與資料一致性必須一起設計

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

練習與參考資料

練習題與詳解分別見本章獨立 PDF。

- Operating System Concepts, 9th Edition，第 7 章，印刷頁 315-346。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與對照範圍

- Silberschatz, Galvin, and Gagne, *Operating System Concepts*, 9th edition, Chapter 7, printed pages 315-339.
  - 7.1 System Model，printed pages 315-317：resource type、request/use/release state 與 deadlocked set。
  - 7.2 Deadlock Characterization，printed pages 317-322：四必要條件、resource-allocation graph 的 edge 定義與 cycle 判準。
  - 7.3 Methods for Handling Deadlocks，printed page 322：prevention、avoidance、detection/recovery 與忽略策略。
  - 7.4 Deadlock Prevention，printed pages 323-327：四條必要條件的破壞方法、資源排序反證與動態鎖問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與對照範圍 (續)

- Silberschatz, Galvin, and Gagne, *Operating System Concepts*, 9th edition, Chapter 7, printed pages 315-339.
  - 7.5 Deadlock Avoidance，printed pages 327-333：safe state、claim edge、Banker safety algorithm 與 request algorithm。
  - 7.6 Deadlock Detection，printed pages 333-337：wait-for graph、多 instance reduction 與偵測頻率。
  - 7.7 Recovery from Deadlock，printed pages 337-338：process termination、resource preemption、victim、rollback、starvation。
  - 7.8 Summary，printed page 339。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study